

Techniques Behind OpenAI o1: Guesses and Thoughts

Presenter: Hongming Zhang

Date: 2024.12.18

OpenAI o1



Introduction

A new large language model trained with reinforcement learning to perform complex reasoning. o1 thinks before it answers—it can produce a long [internal chain of thought](#) before responding to the user.

OpenAI o1



Introduction

A new large language model trained with reinforcement learning to perform complex reasoning. o1 thinks before it answers—it can produce a long internal chain of thought before responding to the user.

Technical roadmap

Through **reinforcement learning**, o1 learns to hone its **chain of thought** and refine the strategies it uses.

- It learns to recognize and correct its mistakes.
- It learns to break down tricky steps into simpler ones.
- It learns to try a different approach when the current one isn't working

OpenAI o1



Introduction

A new large language model trained with reinforcement learning to perform complex reasoning. o1 thinks before it answers—it can produce a long internal chain of thought before responding to the user.

Technical roadmap

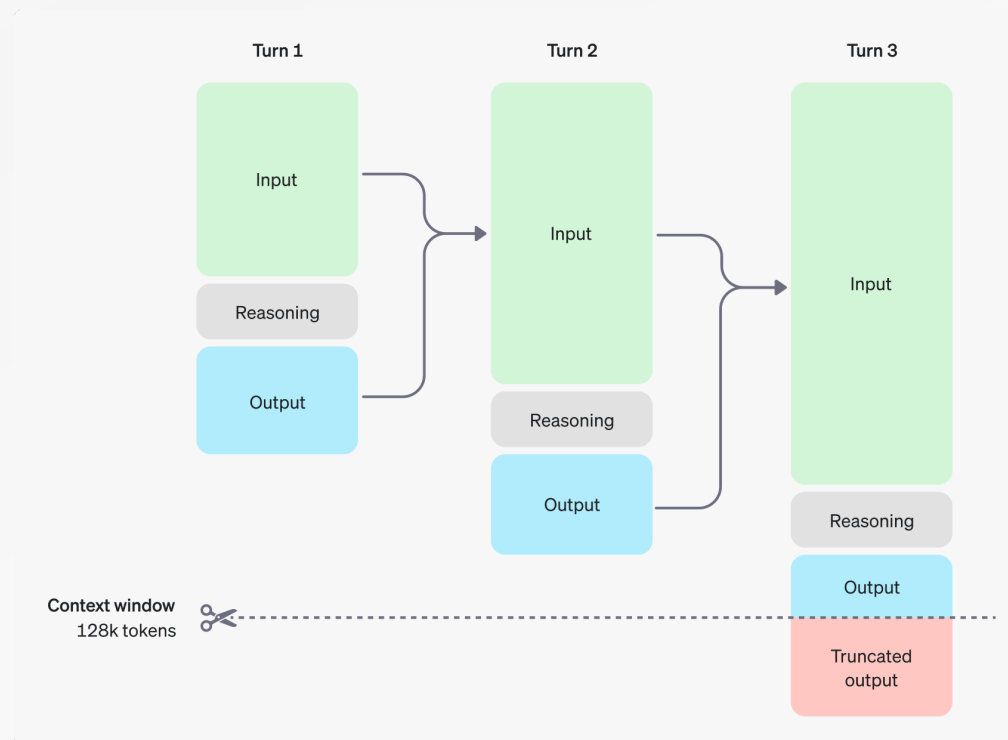
Through reinforcement learning, o1 learns to hone its chain of thought and refine the strategies it uses.

- It learns to recognize and correct its mistakes.
- It learns to break down tricky steps into simpler ones.
- It learns to try a different approach when the current one isn't working

Naming

For complex reasoning tasks this is a [significant advancement](#) and represents a new level of AI capability. Given this, we are [resetting the counter](#) back to 1 and naming this series OpenAI o1.

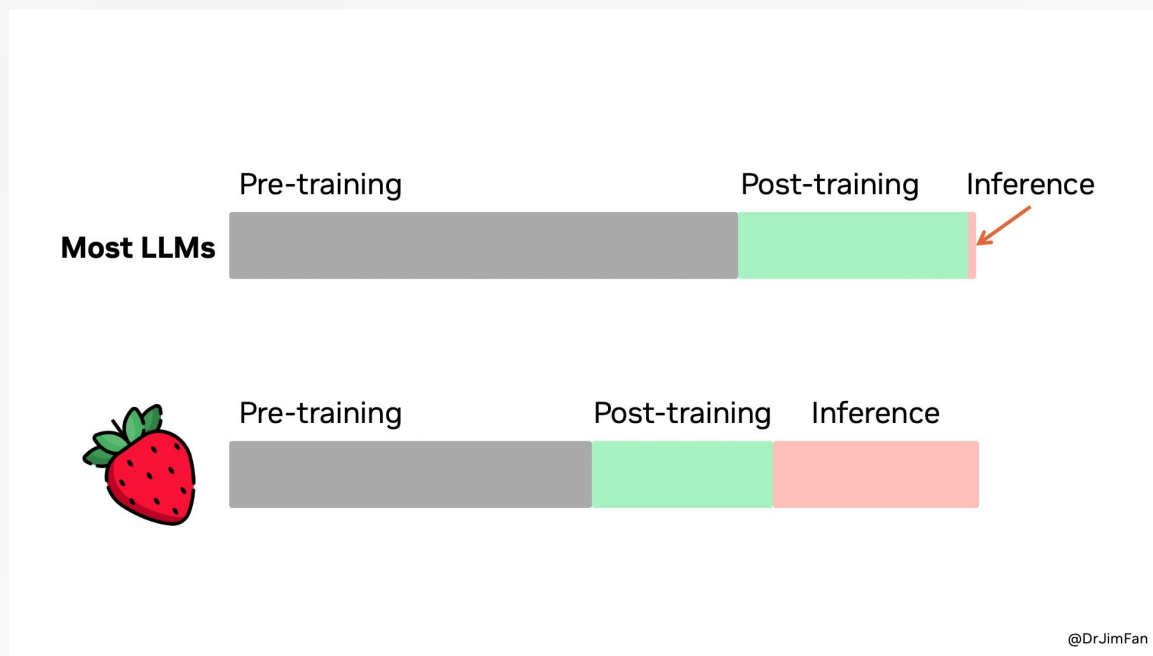
High-Level Idea



The o1 models introduce reasoning tokens

- The models use these reasoning tokens to "think", breaking down their understanding of the prompt and considering multiple approaches to generating a response.
- After generating reasoning tokens, the model produces an answer as visible completion tokens, and discards the reasoning tokens from its context.

Difference Between o1 and Previous Models



As Sutton said in [the Bitter Lesson](#), there're only 2 techniques that scale indefinitely with compute: [learning & search](#). It's time to shift focus to the latter.

[LLMs are text-based simulators](#). By [rolling out](#) many possible strategies and scenarios in the simulator, the model will eventually converge to good solutions. The process is a well-studied problem like AlphaGo's monte carlo tree search (MCTS).

Difference Between o1 and Previous Models

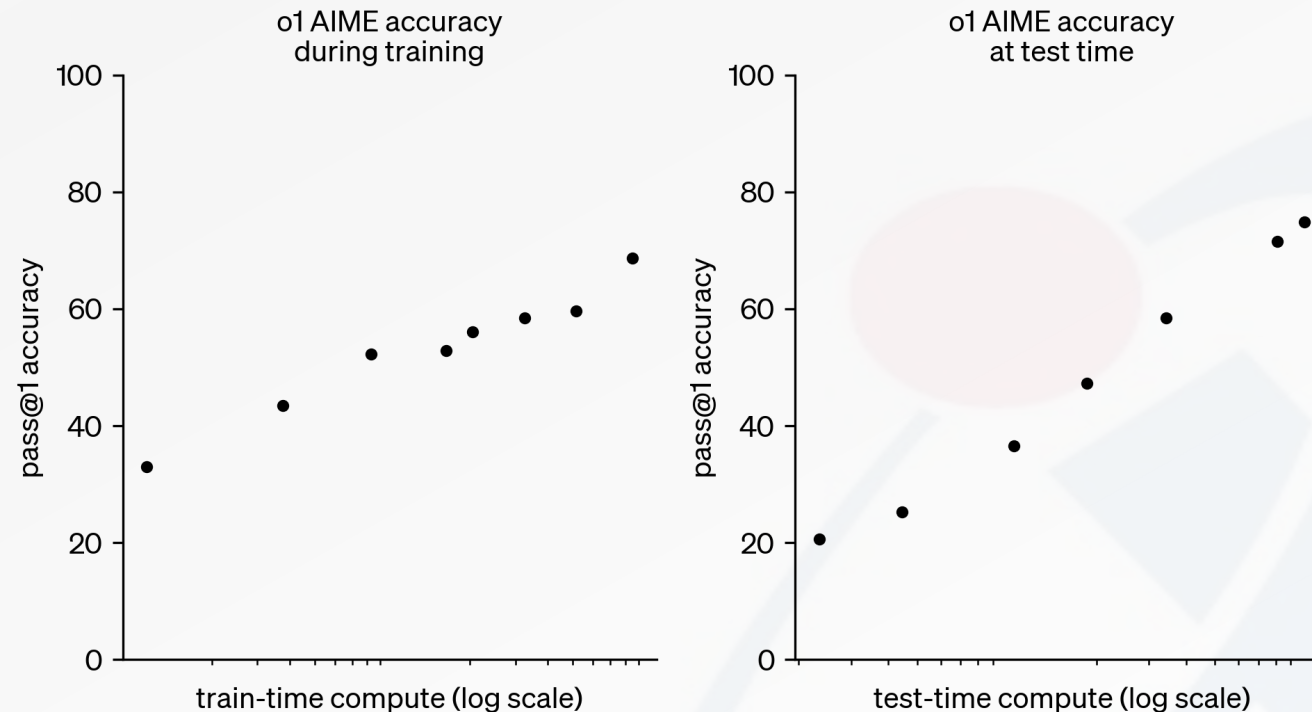


The key difference between o1's training and that of previous models (GPT4, ChatGPT, GPT4-o) is predominantly the nature of the **training data** and **rewarding the intermediate steps** leading to the solution.

In previous iterations, models were primarily trained on (prompt, solution) pairs: given a problem as input, the model was expected to produce the **final answer as output**.

In contrast, o1 is trained on input problems paired with **detailed, step-by-step explanations** of how to reach the solution and each step of the solution is then scored.

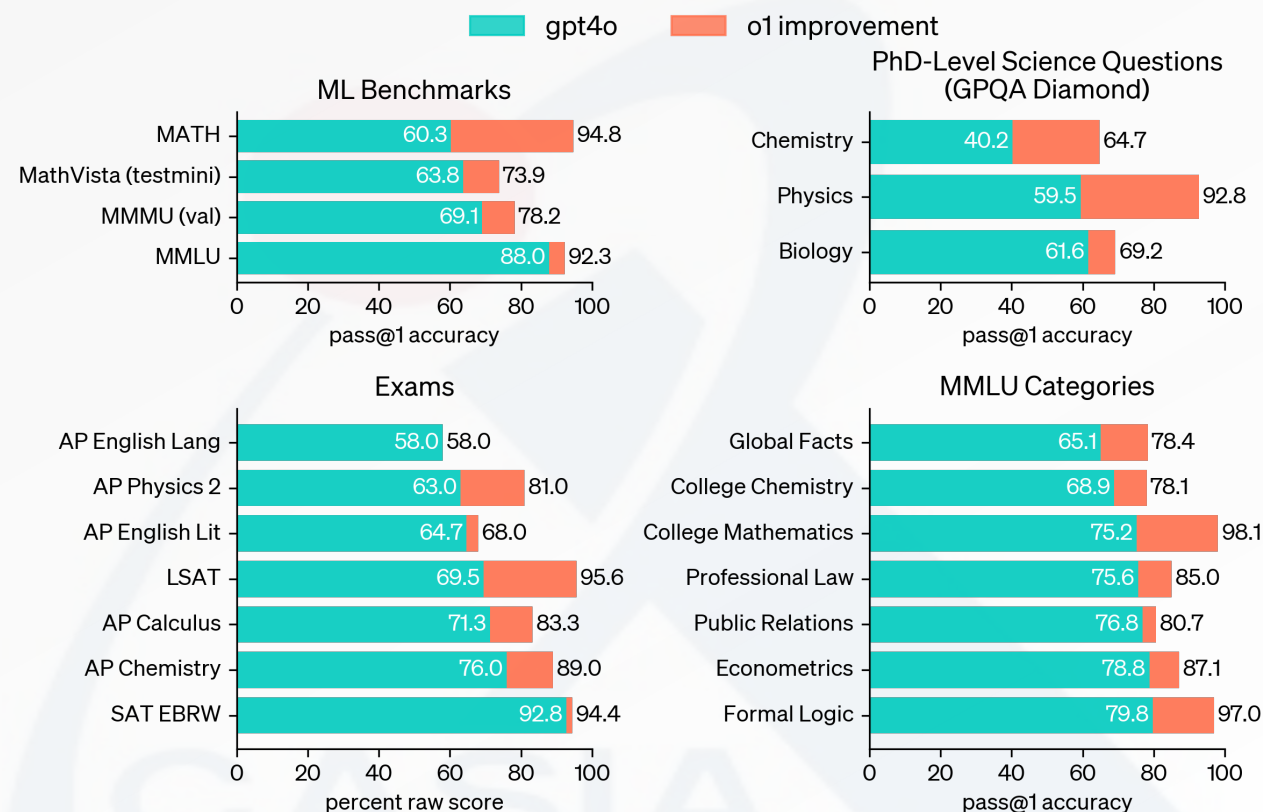
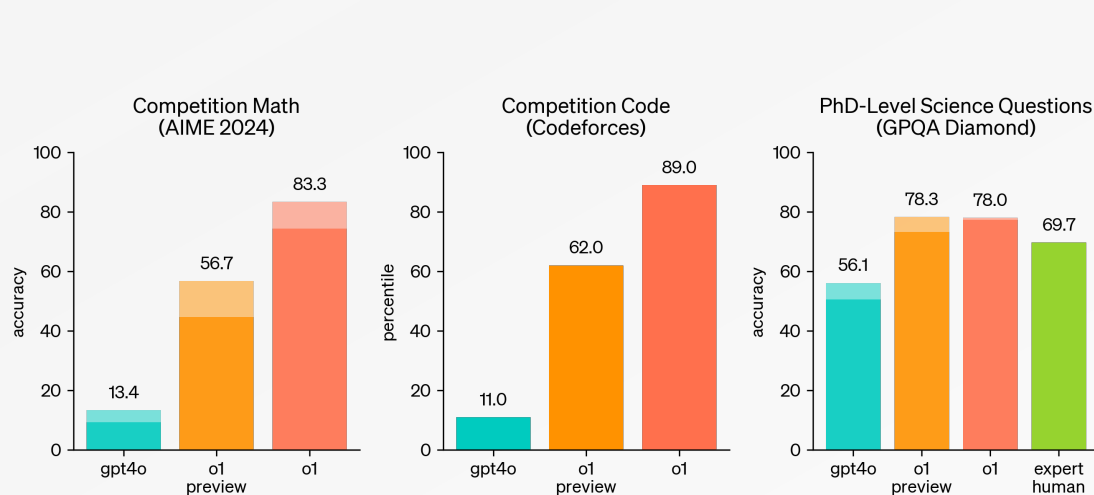
Scaling Law



o1 performance smoothly improves with both train-time and test-time compute

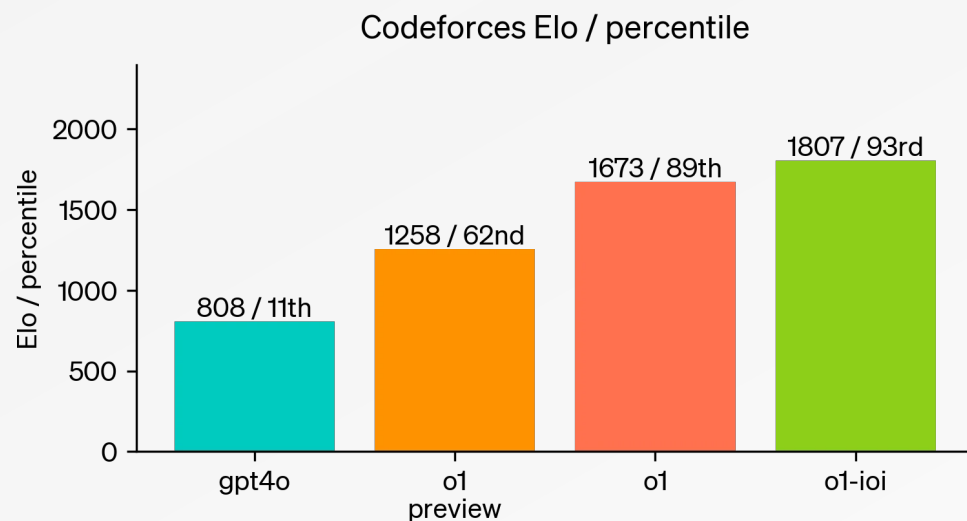
Results

o1 significantly outperforms GPT-4o on the vast majority of these reasoning-heavy tasks.

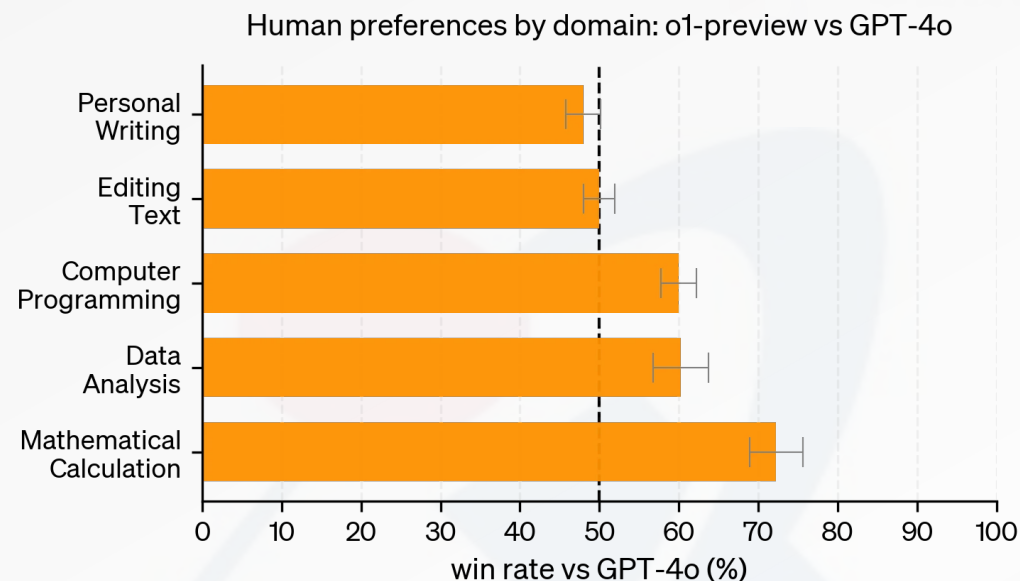


Solid bars show pass@1 accuracy and the shaded region shows the performance of majority vote (consensus) with 64 samples.

Results



Further fine-tuning on programming competitions improves o1



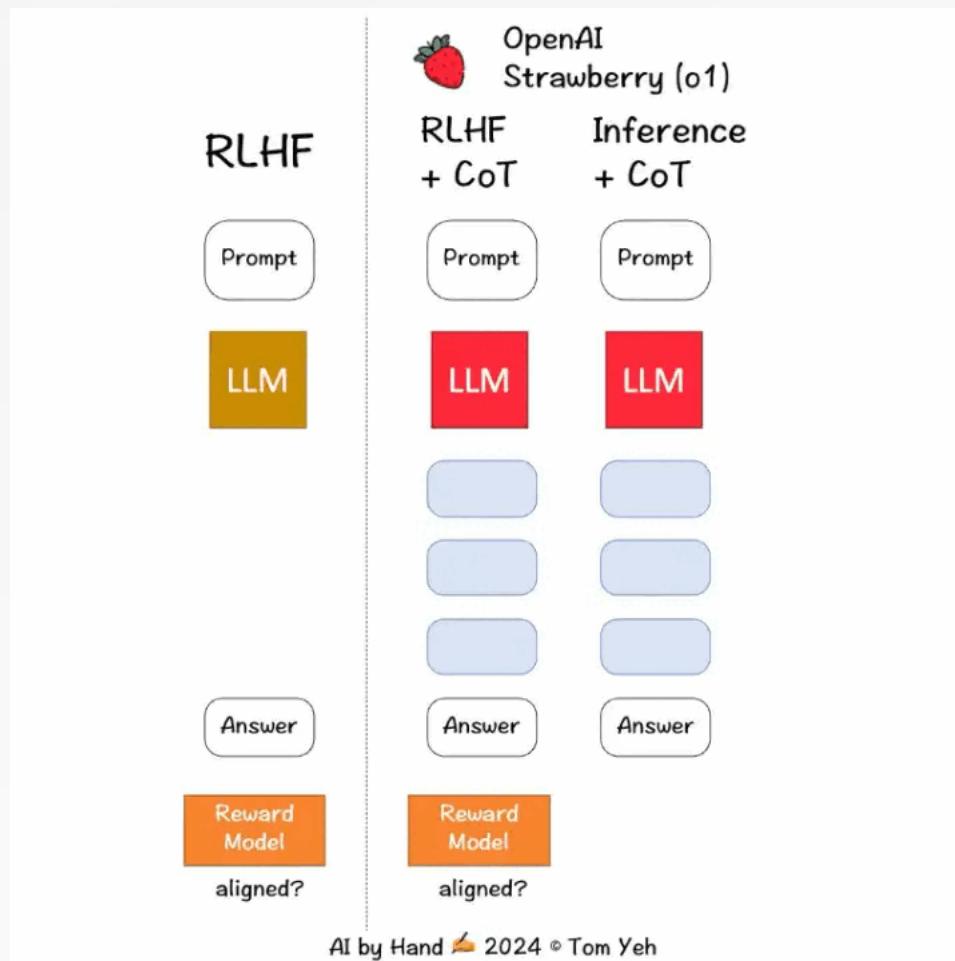
o1-preview is preferred to gpt-4o by a large margin in reasoning-heavy categories like data analysis, coding, and math.

o1-preview is not preferred on some natural language tasks, suggesting that it is not well-suited for all use cases.

Techniques Behind OpenAI o1

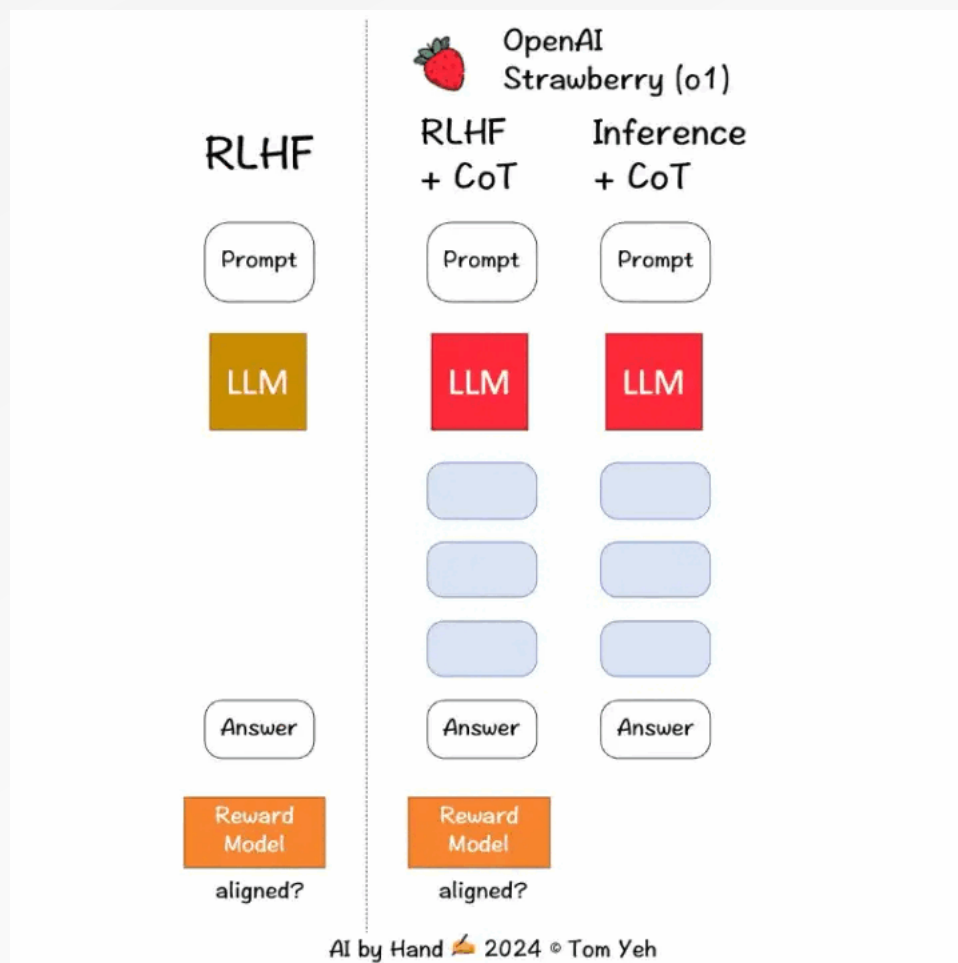
Pre-training? + Post-training + Inference

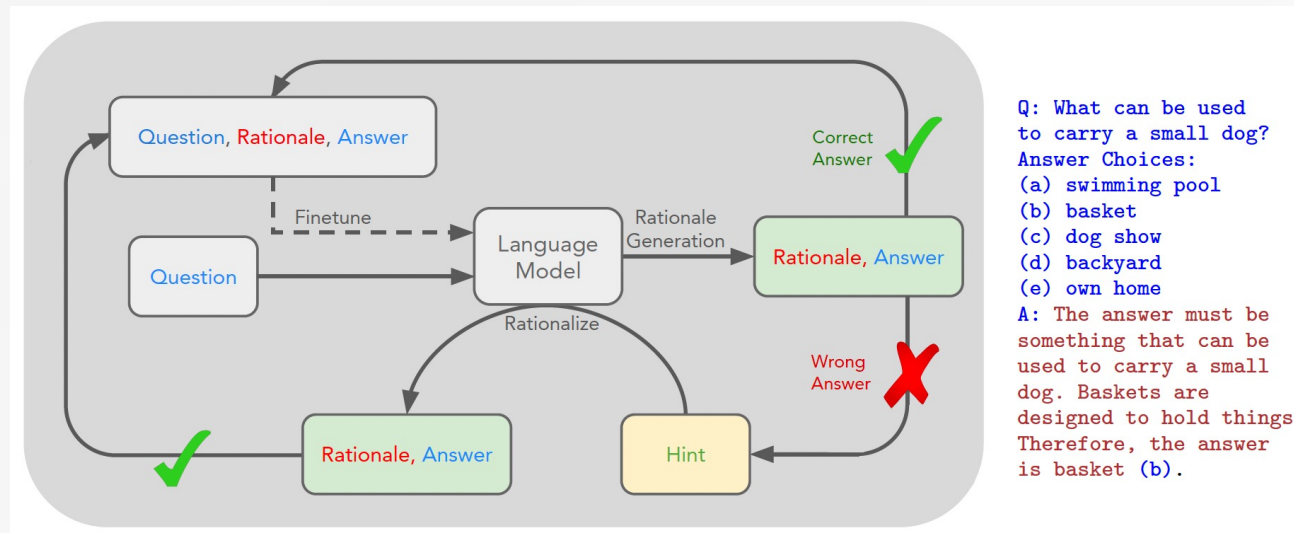
Techniques Behind OpenAI o1



Pre-training? + Post-training + Inference

Post-training





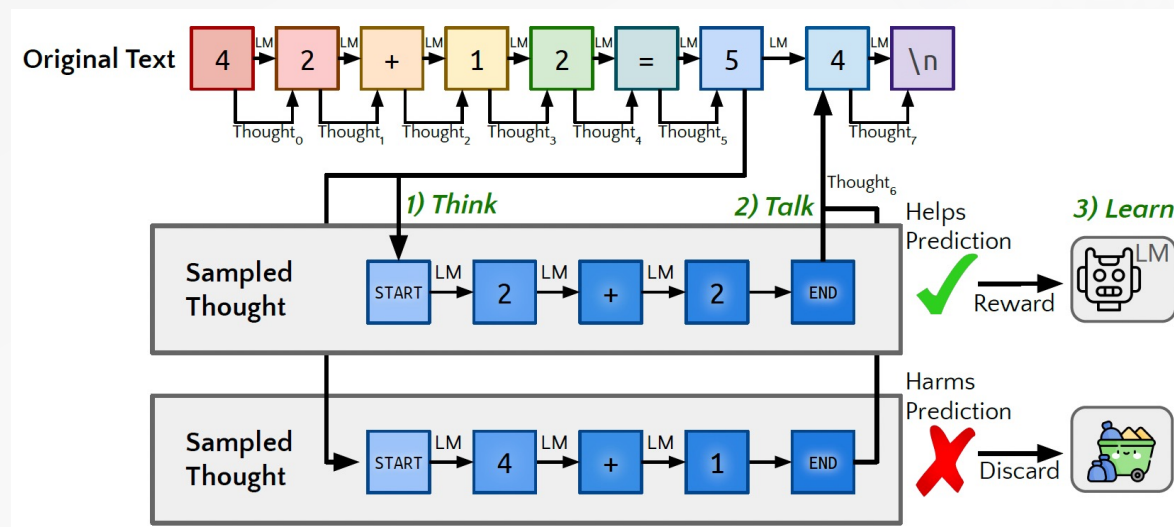
Prompt a large language model to **self-generate rationales** and **refine** the model's ability further by fine-tuning on those rationales that lead to correct answers.

- Generate rationales to **answer** many questions, prompted with a few rationale examples;
- If the generated answers are **wrong**, **try again** to generate a rationale **given the correct answer**;
- **Finetune** on all the rationales that ultimately yielded correct answers;

Improvements in rationale generation improve the training data, and improvements in training data further improve rationale generation.

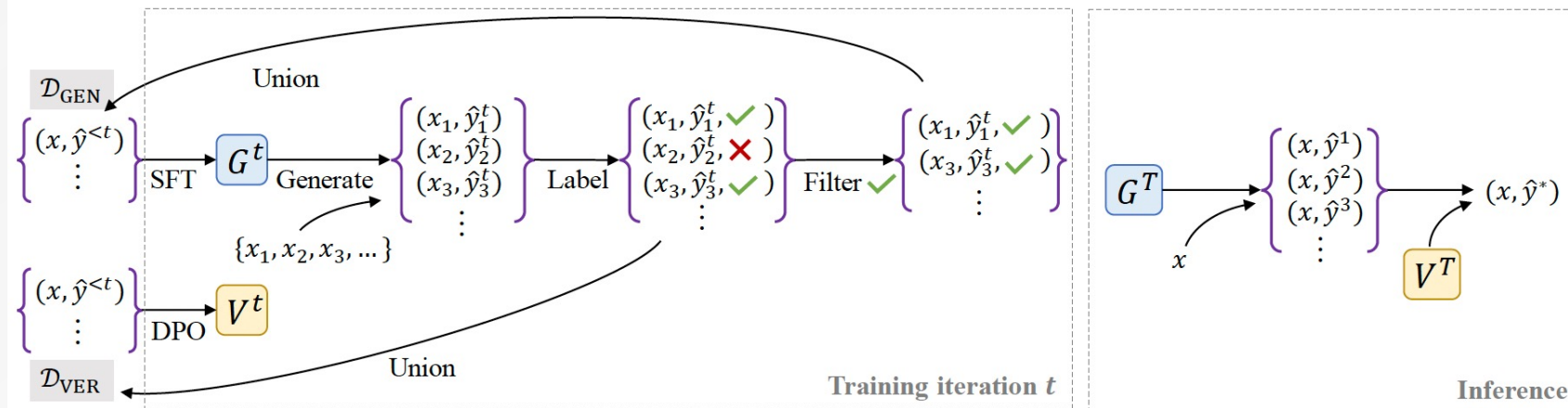
Quiet-STaR

Generalize STaR to learn reasoning from diverse **unstructured text data**.



Train an LM to generate reasoning at each token that helps it **infer future text** from a large internet text corpus

- Generate thoughts, in parallel, following all tokens in the text (think).
- Apply REINFORCE, as in STaR, to **increase the likelihood** of thoughts that help the model predict future text while **discarding** thoughts that make the future text less likely (learn).

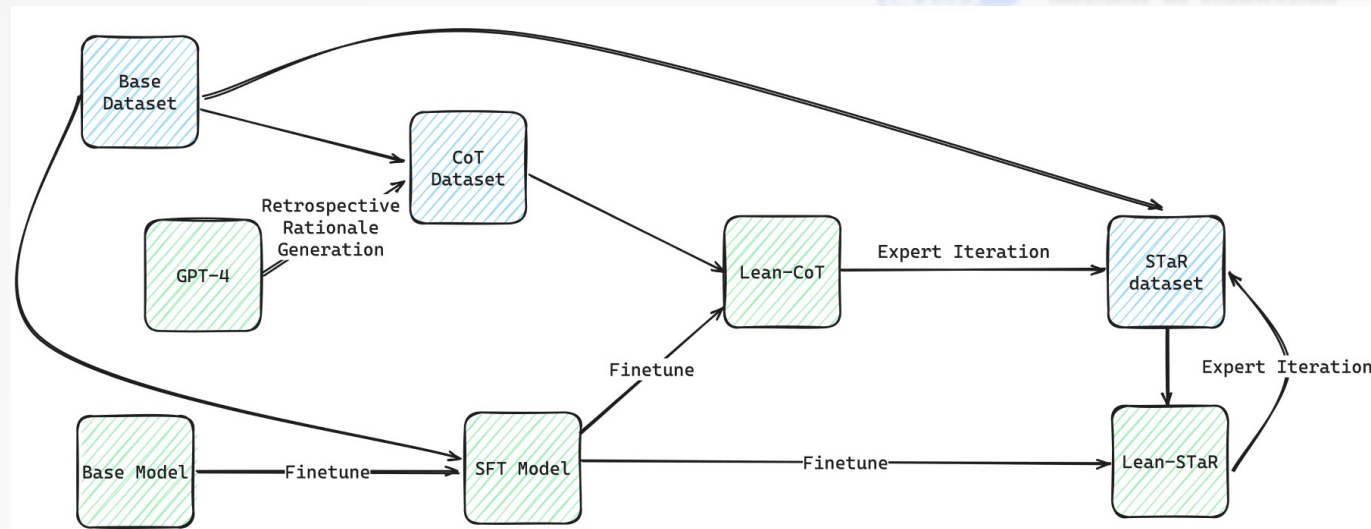


Previous approaches discard incorrect solutions generated, neglecting valuable information.

- Utilize both the correct and incorrect solutions generated during the self-improvement process to train a **verifier** using DPO that judges correctness of model-generated solutions.
- This verifier is used at inference time to select one solution among many candidate solutions.

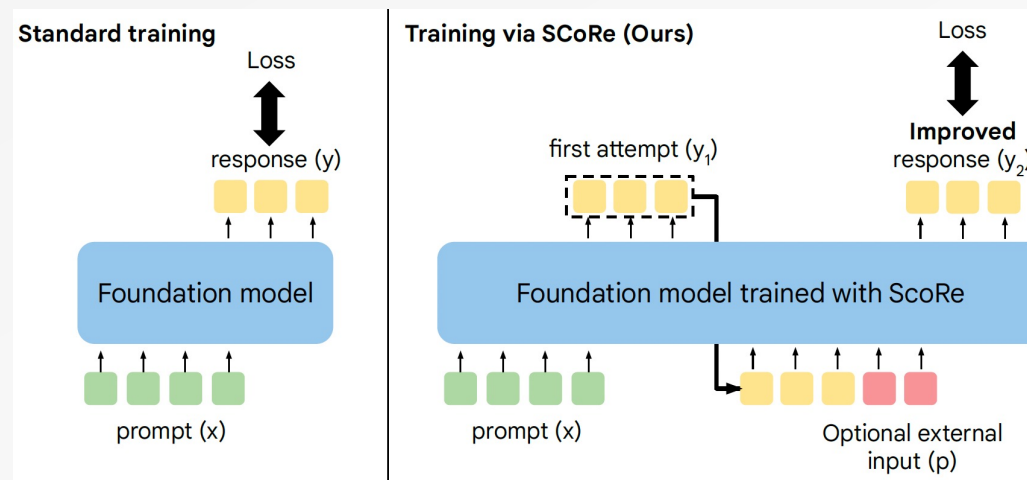
Lean-STaR

Training language models to produce **informal thoughts** prior to each step of a proof, boosting the model's **theorem-proving capabilities**.

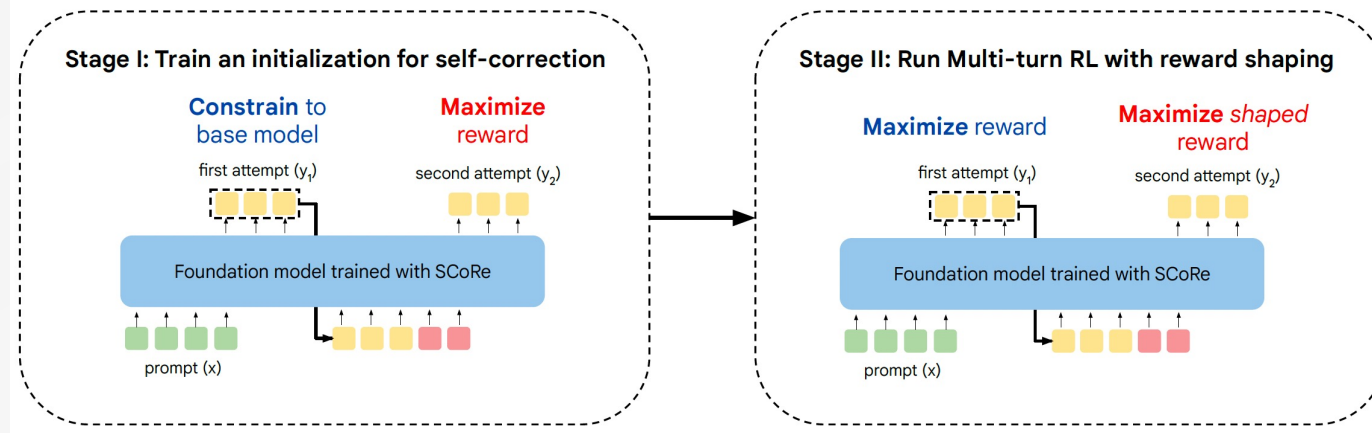


A wealth of informal information that is not present in formal proofs can be useful for learning to prove theorems

- Produce **CoT dataset** through GPT-4.
- Fine-tune the SFT model with the CoT dataset to obtain **Lean-CoT**.
- Use expert iteration to generate the **STaR dataset** through the model in the last iteration (Lean-CoT in the first iteration) and then **fine-tune** Lean-CoT on the updated STaR dataset to obtain the model in the next iteration



- Distribution shift. Able to correct errors made by the base model that generated the data, **do not transfer** to self-correction under the learned model's own mistakes
- Behavior collapse. Best first-attempt response followed by superficial or **no modifications in the second attempt**



Stage I: train an initialization that can **produce high-reward responses in the second-attempt** while mimicking the base model's initial response at the first attempt

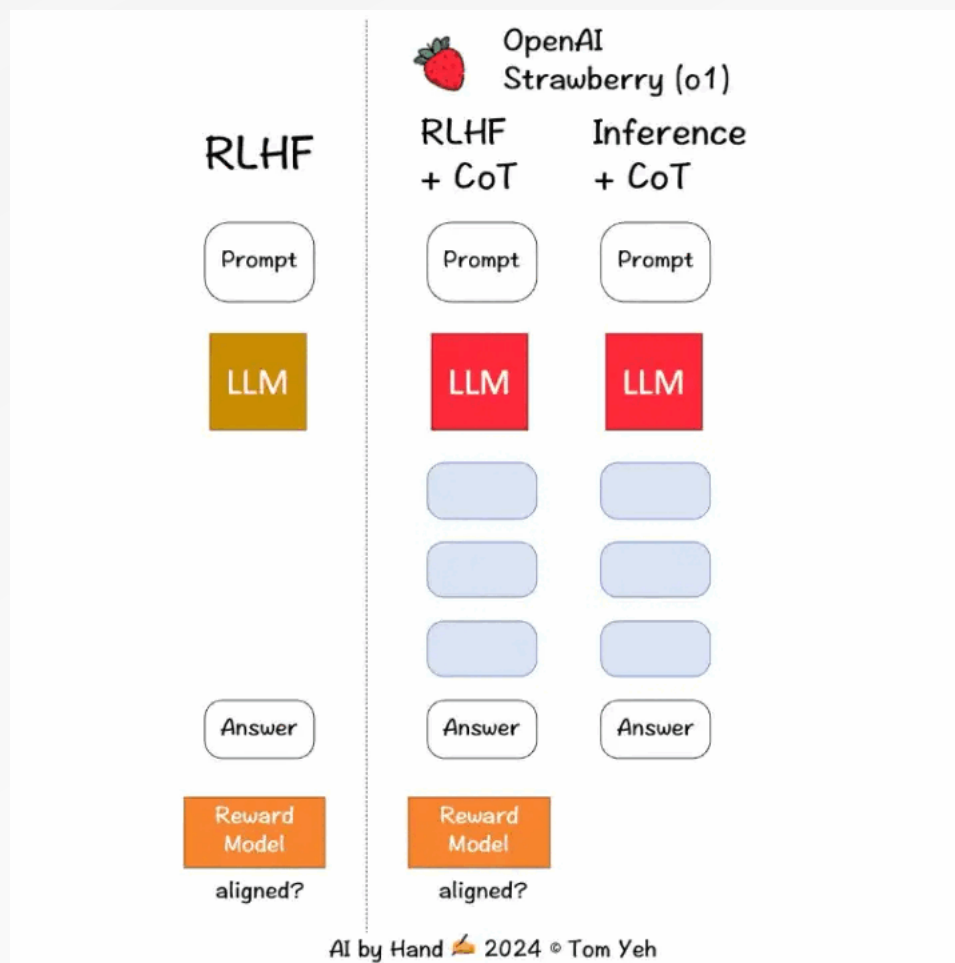
$$\max_{\theta} \mathbb{E}_{\mathbf{x}_1, \mathbf{y}_1 \sim \pi_{\theta}(\cdot|\mathbf{x}), \mathbf{y}_2 \sim \pi_{\theta}(\cdot|[\mathbf{x}_1, \mathbf{p}_1])} \left[\hat{r}(\mathbf{y}_2, \mathbf{y}^*) - \beta_2 D_{KL}(\pi_{\theta}(\cdot|\mathbf{x}_1) || \pi_{\text{ref}}(\cdot|\mathbf{x}_1)) \right], \quad (3)$$

Stage II: **jointly optimizing both attempts**, a **shaped reward** to incentivize the discovery of the **self-correction strategy** instead of the simple strategy of producing the best first response followed by making any minor edits to it in the second attempt

$$\max_{\theta} \mathbb{E}_{\mathbf{x}_1, \mathbf{y}_1 \sim \pi_{\theta}(\cdot|\mathbf{x}), \mathbf{y}_2 \sim \pi_{\theta}(\cdot|[\mathbf{x}_1, \mathbf{p}_1])} \left[\sum_{i=1}^2 \hat{r}(\mathbf{y}_i, \mathbf{y}^*) - \beta_1 D_{KL}(\pi_{\theta}(\cdot|\mathbf{x}_i) || \pi_{\text{ref}}(\cdot|\mathbf{x}_i)) \right], \quad (4)$$

the second attempt with a bonus $\alpha \cdot (\hat{r}(\mathbf{y}_2, \mathbf{y}^*) - \hat{r}(\mathbf{y}_1, \mathbf{y}^*))$

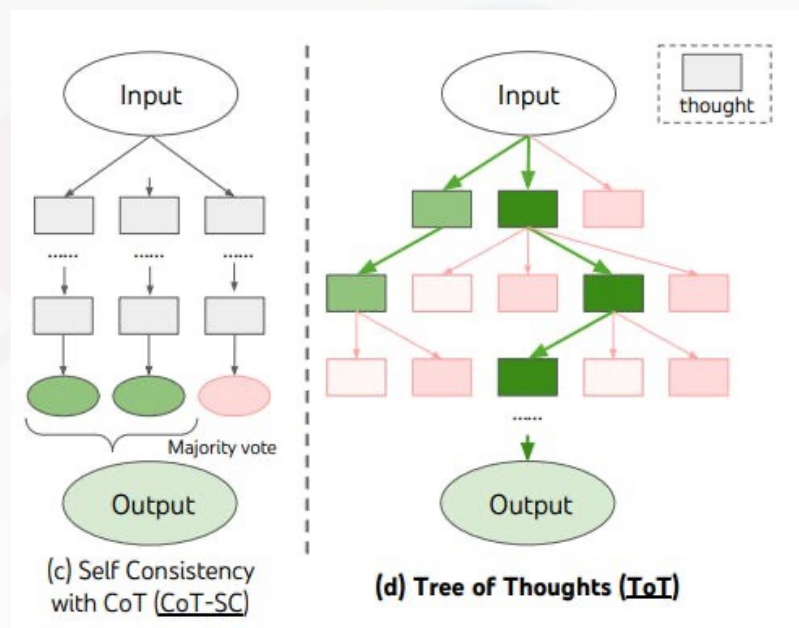
Inference



Scoring in LLMs

Consensus: generating multiple candidates and selecting the most common one (**majority voting**)

- easily implemented when you need to return a **single number**
- **challenging to achieve consensus when writing a proof**, as it's unlikely that the model will generate the same proof multiple times.



Scoring in LLMs

Reward model: multiple solutions are sampled and then scored using a reward model. The solution with the **highest score** is returned.

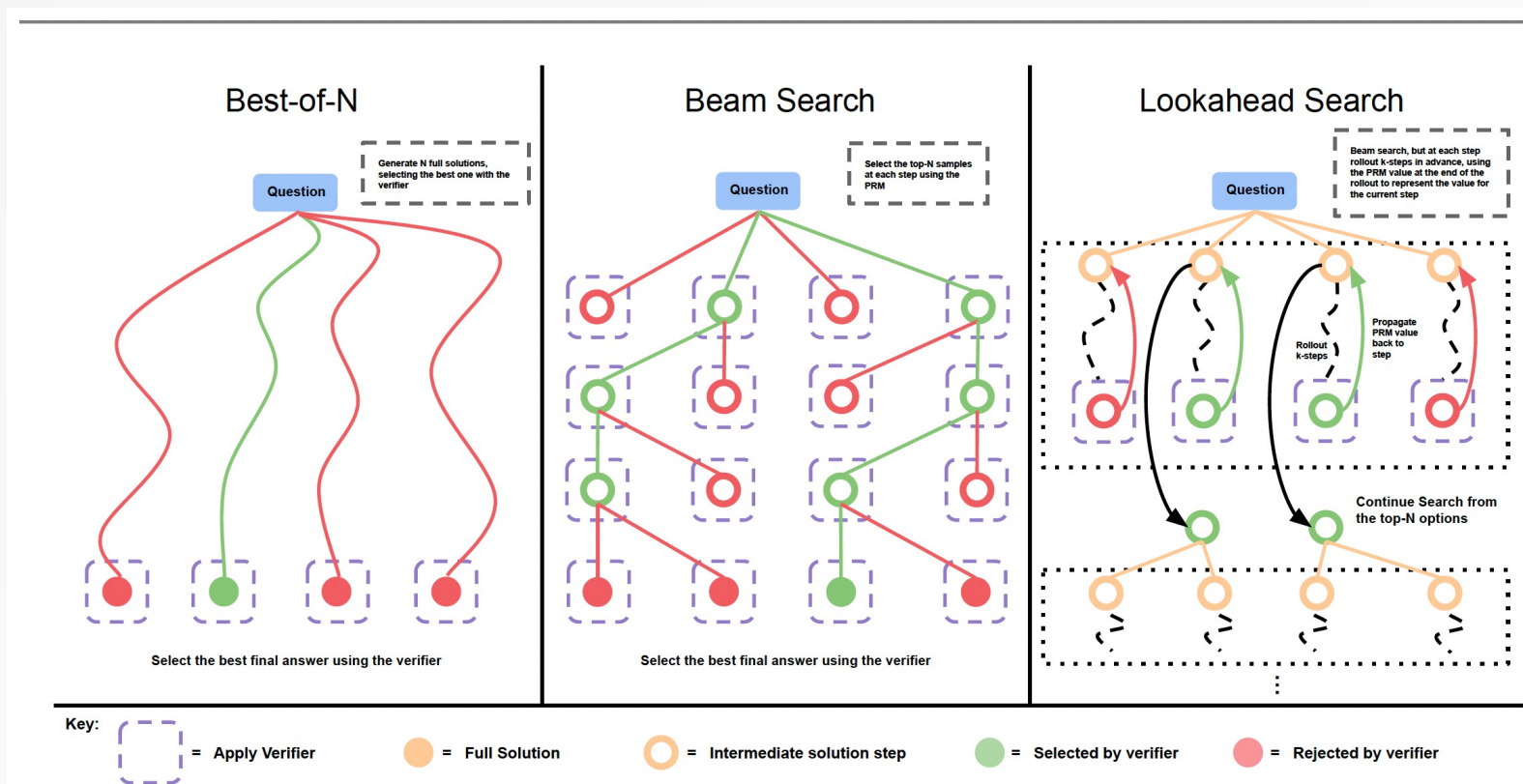
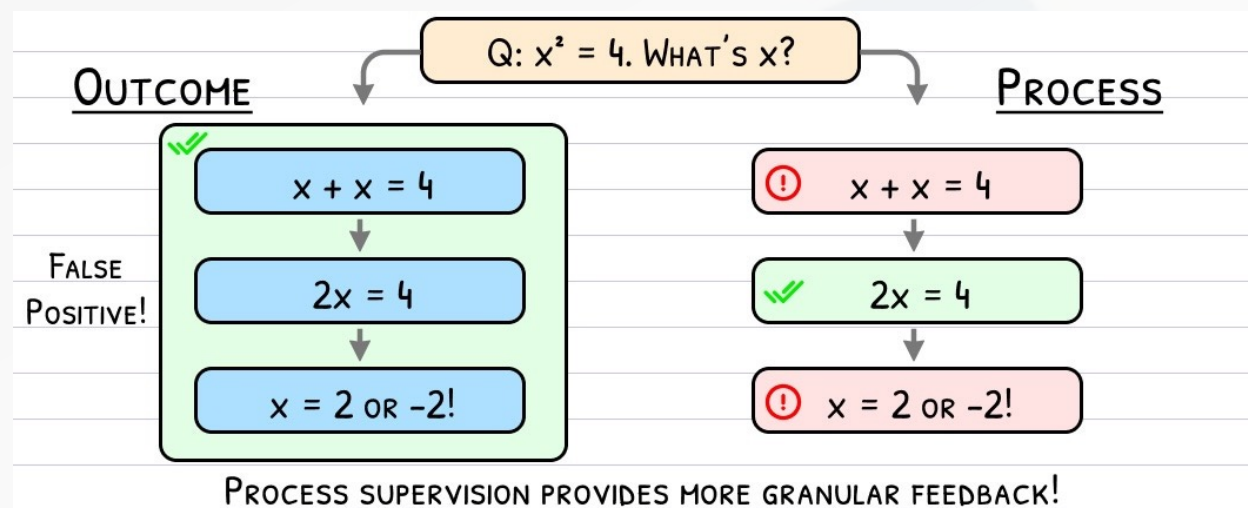


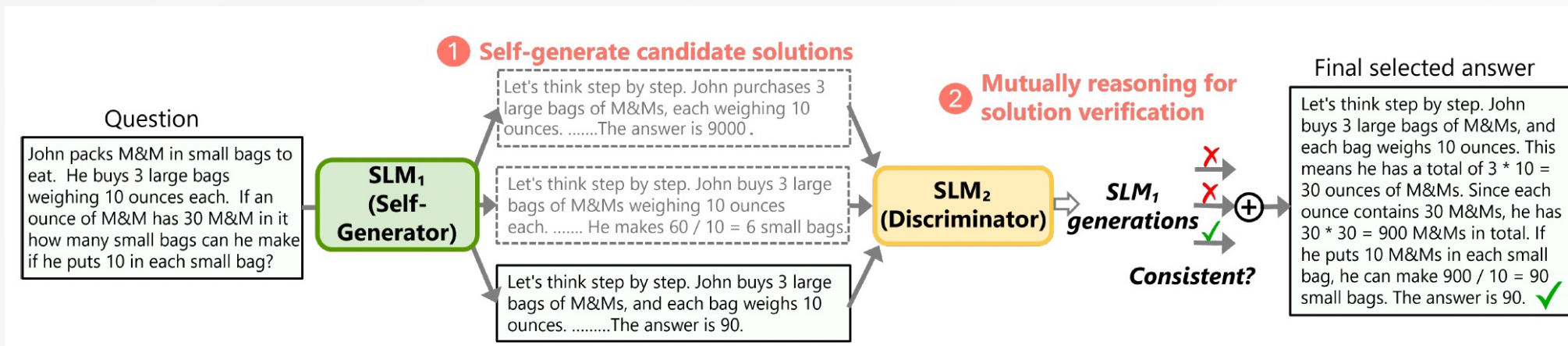
Figure 2 | **Comparing different PRM search methods.** **Left:** Best-of-N samples N full answers and then selects the best answer according to the PRM final score. **Center:** Beam search samples N candidates at each step, and selects the top M according to the PRM to continue the search from. **Right:** lookahead-search extends each step in beam-search to utilize a k-step lookahead while assessing which steps to retain and continue the search from. Thus lookahead-search needs more compute.

Scoring in LLMs

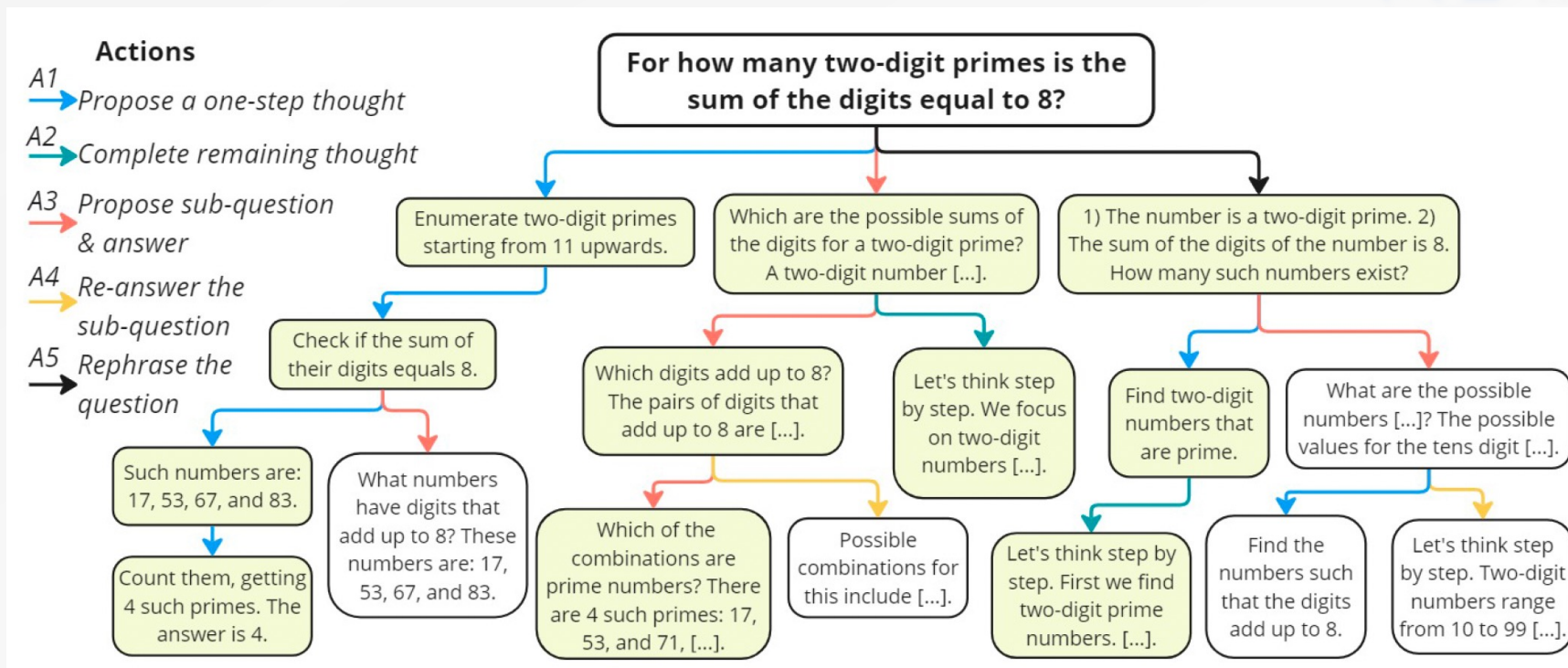
Process Reward Models (PRM): **verifying every step** individually rather than the entire sample



- PRMs provide a **richer signal**, more effective than both outcome reward models and majority voting
- No simple way to automate process supervision; it relies on **human data labelers** or large-scale models



Improves reasoning capabilities of small language models (SLMs) *without finetuning or superior models*



- A target SLM augments the Monte Carlo Tree Search (MCTS) with a rich set of human-like reasoning actions to construct higher quality reasoning trajectories.
- Another SLM, with capabilities similar to the target SLM, acts as a discriminator to verify each trajectory generated by the target SLM.

Reference

<https://openai.com/index/introducing-openai-o1-preview/>

<https://openai.com/index/learning-to-reason-with-llms/>

<https://platform.openai.com/docs/guides/reasoning>

<https://www.zhihu.com/question/666999747/answer/4472268952>

<https://www.zhihu.com/question/666999747/answer/3633105859>

<https://www.zhihu.com/question/666999747/answer/11708900226>

<https://substack.com/home/post/p-148782195>

<https://zhuanlan.zhihu.com/p/773907223>

<https://zhuanlan.zhihu.com/p/864190605>

<https://zhuanlan.zhihu.com/p/905620136>

<https://zhuanlan.zhihu.com/p/926807168>

<https://x.com/DrJimFan/status/1834279865933332752>

<https://x.com/ProfTomYeh/status/1834617696215806285>

<https://github.com/hijkzzz/Awesome-LLM-Strawberry>



THANKS!

